Calibrated Confidence for Kilometre-Scale Feed-Forward SLAM

Project Timeline Document, Mobile Robotics

Kavish Vora, Pranav Srivastava, Mehrish Khan
International Institute of Information Technology, Hyderabad

Abstract

Feed-forward 3D models such as VGGT process multiple images jointly and return camera poses, depth estimates, and per-pixel confidence in a single forward pass. Processing thousands of frames jointly is impractical at the memory budgets available to us. VGGT-Long therefore processes the sequence in overlapping windows and connects neighbouring windows using Sim(3) alignments. This allows the local reconstructions to be chained into trajectories spanning several kilometres, despite the model operating on relatively short windows. In the pose graph, however, every connection between windows is trusted equally, even though some are much worse than others and the network reports a confidence for every pixel. Our project does three things. First, we measure VGGT's confidence against real depth and pose error on KITTI, using LiDAR and ground-truth poses. Second, if the confidence is off, we fit a simple calibration (isotonic regression, no training) that converts it into an uncertainty for every edge of the pose graph. Third, we feed these uncertainties to the VGGT-Long back-end and compare ATE (Sim(3)-aligned RMSE) on KITTI, Waymo and TUM RGB-D against uniform weights and VGGT-Long's own weighting, at the same window size. We also test loop-closure edges and deliberately injected false loops.

1 Motivation

Background. VGGT [1] attends over all frames at once, so its memory grows quickly with the number of frames, and on our GPUs it handles only tens of frames. VGGT-Long [2] gets around this by running the frozen model on overlapping windows. It aligns each window to the next using the frames they share. Each window has its own unknown scale (a single camera cannot measure metres), so the alignment is a similarity transform, Sim(3), not a rigid one. Loops found by place recognition are added, and a pose graph is solved. Each window on its own is accurate. The drift comes from the *seams* between windows. Every seam adds a small rotation, translation and scale error, and a few hundred seams add up to tens of metres. The pose graph is where this error gets spread out and corrected.

We see three shortcomings in the current approach:

S1. All edges are weighted the same. VGGT-Long uses confidence to weight points *inside* each alignment. Its global objective, however, is a plain sum of squared residuals, so every seam and every loop edge gets an identity information matrix, to our reading of [2]. Real seams vary in quality: they get worse with low texture, sharp turns, moving cars and small overlap. The 7 degrees of freedom also differ in reliability, and scale drift is the classic failure of monocular SLAM [3]. Equal weighting gives unreliable edges the same influence on the global solution as well-constrained edges, potentially causing the optimizer to distribute corrections inappropriately.

S2. Confidence has not been checked at the level a pose graph needs. DUSt3R, MASt3R and VGGT [1, 4, 5] learn confidence C with a loss of the form $C\,e - \alpha \log C$ [6], where e is the prediction error. This loss is minimised at $C = \alpha/\mathbb{E}[e]$, so a perfectly trained model would satisfy $\mathbb{E}[e \mid C] = \alpha/C$. Existing audits work per pixel: Hillemann et al. [7] on DTU, and the calibration audit of N. Deng et al. [8] on seven backbones and 13 datasets including KITTI, who find that confidence ranks error well but understates its magnitude and propose a two-constant power-law correction. To our knowledge, no prior work has tested for the quantities a SLAM back-end consumes, namely window-to-window Sim(3) seams and loop closures, or whether correcting it changes the trajectory.

S3. Pixel confidence does not directly give an edge uncertainty. A seam is estimated from about 10^4 pixels. If those pixels are treated as independent and their information is added up, the seam looks far more certain than it is. Their errors are strongly correlated because they come from one forward pass with one shared scale. There is currently no tested way to turn per-pixel confidence into the 7×7 uncertainty a pose graph edge needs.

2 Related Work

Feed-forward geometry. DUSt3R [4] and MASt3R [5] predict 3D pointmaps with confidence for image pairs. VGGT [1] handles many views and also predicts cameras. CUT3R [9] and StreamVGGT [10] process video as a stream with a persistent memory. VGGT-Ω [11] adds scene-summarising register tokens and trains with less memory, but to our knowledge is not evaluated on long driving sequences.

Long sequences. VGGT-Long [2], described above, is our baseline. VGGT-SLAM [12] argues that Sim(3) alignment is not enough for uncalibrated cameras and aligns submaps with 15-DOF SL(4) transforms instead. We keep Sim(3) to match our baseline; our weighting method would carry over to SL(4) unchanged. MASt3R-SLAM [13] uses MASt3R confidences as weights in its back-end, but does not check whether they are calibrated. DROID-SLAM [14] learns its confidence weights end to end, which needs training. Sim(3) pose graphs for monocular scale drift go back to Strasdat et al. [3].

Uncertainty and robust back-ends. Kuleshov et al. [15] measure calibration of regression models with reliability diagrams and fix it with isotonic regression; we follow their method. The calibration audit of N. Deng et al. [8] (not the VGGT-Long authors) checks native confidence on seven feed-forward backbones and 13 datasets, finding good error ranking but miscalibrated uncertainty magnitude and proposing a post-hoc power-law correction. DCS [16] protects pose graphs against wrong loop closures, and we reuse its false-loop testing protocol.

3 Contributions

Each contribution addresses one shortcoming from Section 1:

C1. Confidence audit on outdoor driving data (addresses S2). We will extend pixel-level audits to window-to-window seams and loop closures, while retaining the per-pixel LiDAR check as a replication. We will compare VGGT's confidence with real error at both levels, using ground-truth relative poses for edges. We will publish reliability diagrams and test the $\mathbb{E}[e \mid C] \propto 1/C$ relation directly.

C2. Calibrated edge uncertainties without training (addresses S1 and S3). We will learn a simple monotone map from seam-level confidence statistics to a standard deviation for rotation, translation and scale. Together these give a 7×7 information matrix for every seam and loop edge. The map is fitted in seconds on a CPU and plugs into the VGGT-Long back-end without other changes.

C3. Systematic evaluation across domains and failure modes. We will compare seven weighting schemes, three robust kernels, and loops on and off, at equal window size on held-out KITTI sequences, Waymo and TUM RGB-D. The comparison also includes ATE against sequence length, a false-loop stress test, and a memory and speed study on 8–11 GB GPUs. Since the method only reads cached model outputs, other engines can be dropped in: as bonus experiments we test streaming models (CUT3R, StreamVGGT) and VGGT-Ω (Step 7).

4 Proposed Methodology

Step 1: Reproduce the baseline. We split each sequence into windows of W frames with stride S, so neighbouring windows share $W - S$ frames. Each window goes through pretrained VGGT once,

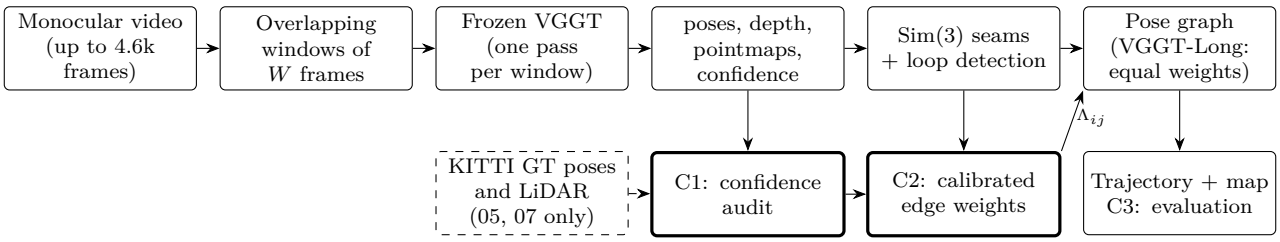


Figure 1: Pipeline. The top row is VGGT-Long, which we reproduce. Thick boxes are our additions. Ground truth (dashed) is used only on the fitting sequences, except by the oracle (f).

in fp16 since our Turing GPUs lack bfloat16, and all outputs are cached to disk. All loop candidates, including those obtained with the lowered place-recognition threshold used for the realistic false-loop test, are also cached up front. After that, every experiment runs on the CPU. Images are resized without distorting the aspect ratio, and camera intrinsics are scaled to match. For two neighbouring windows, the same pixel in a shared frame gives a 3D point in each window, which provides thousands of point pairs. The seam $T_{k,k+1}$ is the Sim(3) transform that best maps one set onto the other. It is found with RANSAC and a confidence-weighted Umeyama fit [17], as in VGGT-Long. Loops are found by comparing one global feature per image from a DINOv2-based encoder [18] (the place-recognition model VGGT-Long uses) and confirmed with the same Sim(3) fit. We keep VGGT-Long's settings for this step, so that only the back-end differs between methods.

Step 2: Pose graph. Each window k has a pose $S_k \in \mathrm{Sim}(3)$ in the world. An edge with measurement Z_{ij} says $S_j = S_i Z_{ij}$, and its error is the 7-vector

$$r_{ij} = \mathrm{Log}\,(Z_{ij}^{-1} S_i^{-1} S_j) = [\,\text{rotation (3)},\ \text{translation (3)},\ \text{log-scale (1)}\,].$$

We minimise $\sum \rho(r_{ij}^\top \Lambda_{ij} r_{ij})$ with Levenberg–Marquardt, where Λ_{ij} is the edge's information matrix (inverse covariance) and ρ is an optional robust kernel. The first window is held fixed. Note that multiplying every Λ by the same constant does not change the solution; only relative weights matter. To make the comparison fair, every weighting scheme is scaled to the same average weight.

Step 3: Measure the real errors (C1). On the fitting sequences, we compute the true relative seam from the full ground-truth poses, using both position and orientation rather than camera positions alone. This gives the true seam, and the seam error splits into a rotation, translation and scale part. Loop-closure edges are measured the same way, since a wrong loop is what folds the map. For depth, we project LiDAR points into the image, rescale the predicted depth with one factor per window, and compute the relative depth error per pixel. We then sort pixels (and seams) by confidence into equal-sized bins and plot each bin's real error against its confidence. This is a reliability diagram [15]. We also fit a line to log error against $\log C$ and check whether its slope is -1, as the training loss predicts. Rank correlation (Spearman) is reported once, but it cannot show an improvement after calibration because a monotone map leaves the ranking unchanged.

Step 4: Calibrated edge uncertainty (C2). Each seam has a few simple features: the mean confidence of its matched points, the alignment residual, the inlier ratio, the overlap size and the median depth. Separately for rotation, translation and scale, we fit an isotonic (monotone) regression from the chosen feature to the expected squared error on the fitting sequences. The result gives three standard deviations, $\sigma_r, \sigma_t, \sigma_s$, and

$$\Lambda_{ij} = \mathrm{diag}(\sigma_r^{-2} I_3,\ (\sigma_t \,\|t_{ij}\|)^{-2} I_3,\ \sigma_s^{-2}).$$

The translation term is relative to the seam length, because each window has its own scale. Loop edges get a separate map, since they fail differently (look-alike places, not slow drift). As a held-out check, we compute the normalized estimation error squared (NEES), $e^\top \Lambda e$, on held-out seams against ground truth; for a calibrated 7-DoF edge its expected value is about 7. NEES uses Λ as fitted, before the rescaling of Step 2, which would change its value.

Step 5: Weighting schemes compared. These are chosen so that we can tell *why* a scheme helps. Per-DoF constants for uniform weights are tuned on the fitting sequences (05/07); robust-kernel

thresholds are tuned separately for each weighting scheme, also on 05/07 only: (a) VGGT-Long's own weighting: confidence weights inside each alignment, identity weights in the graph; (b) uniform weights: the same weight on every edge, with one value each for rotation, translation and scale, which tests whether just rebalancing these helps; (c) raw confidence, $\Lambda \propto \bar{c}^{\,2}$, the training-loss prediction with no calibration; (d) weights from the alignment residual and inlier count, the classical choice; (e) our calibrated weights; (f) an oracle using the true error, which is not a method but shows the most any weighting could gain; and (g) the calibration audit's two-constant power law [8], applied per pixel before edge aggregation and refit on 05/07, since their published VGGT–KITTI constants may include our test sequences. Each scheme is run with no robust kernel, Huber, and DCS [16], and with loops off and on.

Step 6: False-loop stress test. Following [16], we add $k \in \{0, 1, 2, 4, 8, 16\}$ false loops between distant windows, with 20 random seeds. The test has two versions. In the *adversarial* version, every fake loop gets the same high weight, which tests only the robust kernels. In the *realistic* version, we lower the place-recognition threshold until real wrong matches appear, and each scheme weights them by its own rule; this version shows whether calibration helps. A run survives if its trajectory error stays within 1.25× the error with no false loops.

Step 7 (bonus): Streaming models and VGGT-Ω. If time allows: (i) we run CUT3R [9] (fixed-size state) and StreamVGGT [10] (cached keys and values) on the whole sequence in one stream and in windows with our back-end, and compare ATE against sequence length, revisit error and peak GPU memory, to see whether a persistent state removes the need for chunking or only hides the drift. (ii) VGGT-Ω [11] has released checkpoints; if they run on our GPUs, we use the 1B reproduction checkpoint, which its authors retrained for benchmarking. It returns depth, depth confidence, camera poses and per-frame register tokens, so it drops into our pipeline unchanged. It still cannot take a full KITTI sequence in one pass (its authors report 43 GB for 500 frames), to our knowledge is not tested on long driving data, and its confidence has, to our knowledge, not been audited. We (a) run the same confidence audit and check whether the VGGT calibration transfers or must be refit, (b) test its register tokens, which summarise the scene, as the loop-detection feature against the DINOv2-based model, and (c) run the full pipeline with it as the engine at the same memory budget.

5 Datasets

KITTI odometry [19] is our main dataset. Sequences 00–10 have ground-truth poses (271 to 4,661 frames, up to about 5 km), colour images, Velodyne LiDAR and calibration, and VGGT-Long reports results on them. We fix the split before any experiment. The calibration is fitted on **05 and 07**, which have loops. Results are reported on the held-out **00 and 02** (main, long and with loops) and on the other sequences. Leave-one-sequence-out runs check that the result does not depend on the split. The 23.2k frames take one inference pass on the lab 2080 Ti and Kaggle T4 GPUs, after which all experiments run on the CPU.

Waymo Open [20] is our second outdoor dataset. Its segments are about 20 s long (about 200 frames at 10 Hz), with LiDAR and accurate poses. Because the full dataset is very large, we download only the front-camera images, poses and LiDAR for a fixed subset of validation segments, chosen before any experiment (VGGT-Long's segments where they are listed). Waymo adds a second driving domain to the depth audit and tests whether the KITTI calibration transfers.

TUM RGB-D [21] is our indoor dataset: handheld, with motion-capture ground truth (fr1/desk, fr1/room, fr2/xyz, fr3/long_office_household). We use it to test whether the KITTI calibration transfers to a very different setting without refitting.

6 Evaluation Plan

Metrics.

- *Trajectory:* ATE (Sim(3)-aligned RMSE in metres), computed with evo [22]. We also report ATE versus sequence length and KITTI relative errors.
- *Calibration:* calibration curves (predicted against empirical error) for depth, relative pose and loop edges, before and after calibration, expected calibration error, negative log-likelihood, and the fitted slope of $\log e$ against $\log C$.
- *Robustness:* median and spread of ATE against the number of false loops, and how many false loops each scheme survives.
- *Systems:* peak GPU memory, frames per second and solve time for window sizes $W \in \{8, 16, 24, 32\}$.
- *Qualitative:* trajectories against ground truth, point clouds coloured by confidence, and maps folding under false loops.

Baselines. The main comparison is VGGT-Long *re-run by us at the same window size*. Its published numbers (windows of 30–120 frames on 24–48 GB GPUs) are listed separately. We run ORB-SLAM3 (monocular) [23] and MASt3R-SLAM [13] ourselves where they fit in memory, and take DROID-SLAM [14] and CUT3R numbers from published papers, marked as such.

Planned results. Table 1: ATE of all systems on KITTI, Waymo and TUM. Table 2: weighting scheme (including uniform, raw-confidence and calibrated) × robust kernel × loops on the held-out sequences. Table 3: stress test. Table 4: window size against memory, speed and ATE. Figures: calibration curves, ATE against sequence length, and trajectory and map plots.

Fairness rules. All schemes use the same cached outputs, seams, loop candidates, window size and solver settings. The identity-weight solver must reproduce VGGT-Long's own back-end result. Only the weights (and, separately, the robust kernel) change. No ground truth from the test sequences is used in any fitting, except by the oracle (f), an upper bound, not a method. Main configurations are repeated under 3–5 window offsets and reported as mean ± std.

Possible outcomes. (1) Calibrated weights improve ATE or robustness relative to uniform weights and VGGT-Long's weighting. (2) No ATE gain, but the oracle shows how much room weighting has, and the robustness or residual-based results still stand. (3) Raw confidence already ranks seam error well and only its level is off, which the back-end ignores; we then report where calibration still matters, including robust-kernel thresholds, loops versus seams, and DoF balance.

7 Timeline

Dates	Work	Milestone
Oct 4–9	Download KITTI 00–10 and the Waymo subset, set up evo, check LiDAR projection; set up VGGT, 32-frame test, memory per W; pose-graph backend with synthetic tests	**Oct 10:** data, model, conventions verified
Oct 10–16	Run VGGT-Long on KITTI at our W and cache all windows and loop candidates; match its evaluation protocol; mid-sem freeze	**Oct 16:** VGGT-Long reproduced
Oct 17–21	Extract seams and true errors; confidence audit (pixel and seam); fit calibration on 05/07; loop-edge errors	**Oct 22:** audit done
Oct 22–25	Plug calibrated weights into back-end; schemes (a)–(g) on 00/02; first table and figures	**Oct 26:** End-to-end baseline ready; mid submission
Oct 26–Nov 8	Robust kernels × loops; false-loop stress test; Waymo and TUM; ORB-SLAM3 (mono); bonus: CUT3R / StreamVGGT and VGGT-Ω	**Nov 9:** full results, draft to TAs
Nov 9–22	Memory and speed table; MASt3R-SLAM if time; final figures; TA feedback; writing	**Nov 22:** freeze
Nov 23–26	Final report (Nov 23), demo (Nov 26)	**Nov 23 / Nov 26**

References

[1] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, "VGGT: Visual geometry grounded transformer," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2025.

[2] K. Deng, Z. Ti, J. Xu, J. Yang, and J. Xie, "VGGT-Long: Chunk it, loop it, align it: Pushing VGGT's limits on kilometer-scale long RGB sequences," *arXiv preprint arXiv:2507.16443*, 2025.

[3] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale drift-aware large scale monocular SLAM," in *Proc. Robotics: Science and Systems (RSS)*, 2010.

[4] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud, "DUSt3R: Geometric 3D vision made easy," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024.

[5] V. Leroy, Y. Cabon, and J. Revaud, "Grounding image matching in 3D with MASt3R," in *Proc. European Conf. Computer Vision (ECCV)*, 2024.

[6] A. Kendall and Y. Gal, "What uncertainties do we need in Bayesian deep learning for computer vision?," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[7] M. Hillemann, R. Langendörfer, S. Landgraf, and M. Ulrich, "Uncertainty quality of VGGT: An analysis on the DTU benchmark dataset," *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. XI-2-2026, pp. 665–672, 2026. arXiv:2606.16479.

[8] N. N. Deng, Q. Cheng, N. Zeller, and D. Cremers, "A calibration audit of confidence in feed-forward 3D reconstruction models," 2026. arXiv:2608.29705v3.

[9] Q. Wang, Y. Zhang, A. Holynski, A. A. Efros, and A. Kanazawa, "Continuous 3D perception model with persistent state," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2025.

[10] D. Zhuo, W. Zheng, J. Guo, Y. Wu, J. Zhou, and J. Lu, "Streaming 4D visual geometry transformer," *arXiv preprint arXiv:2507.11539*, 2025.

[11] J. Wang, M. Chen, S. Zhang, N. Karaev, J. Schönberger, P. Labatut, P. Bojanowski, D. Novotny, A. Vedaldi, and C. Rupprecht, "VGGT-Ω," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2026. arXiv:2605.15195.

[12] D. Maggio, H. Lim, and L. Carlone, "VGGT-SLAM: Dense RGB SLAM optimized on the SL(4) manifold," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[13] R. Murai, E. Dexheimer, and A. J. Davison, "MASt3R-SLAM: Real-time dense SLAM with 3D reconstruction priors," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2025.

[14] Z. Teed and J. Deng, "DROID-SLAM: Deep visual SLAM for monocular, stereo, and RGB-D cameras," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[15] V. Kuleshov, N. Fenner, and S. Ermon, "Accurate uncertainties for deep learning using calibrated regression," in *Proc. Int. Conf. Machine Learning (ICML)*, 2018.

[16] P. Agarwal, G. D. Tipaldi, L. Spinello, C. Stachniss, and W. Burgard, "Robust map optimization using dynamic covariance scaling," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2013.

[17] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.

[18] M. Oquab, T. Darcet, T. Moutakanni, *et al.*, "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024.

[19] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2012.

[20] P. Sun, H. Kretzschmar, X. Dotiwalla, *et al.*, "Scalability in perception for autonomous driving: Waymo open dataset," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020.

[21] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012.

[22] M. Grupp, "evo: Python package for the evaluation of odometry and SLAM." https://github.com/MichaelGrupp/evo, 2017.

[23] C. Campos, R. Elvira, J. J. Gómez Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.